

Netcompany



C0200 - User Guide - Agentic AI Guidelines

Netcompany A/S
AI Transition

Version	0.1
Status	01 - Planned
Approver	[Reviewer]
Author	[Author]

© Copyright 2026 Netcompany
All rights reserved.

Document history

Version	Date	Author	Status	Comments
1.0	29-05-2026	Alexander Colliander Hansen	Completed	
1.1	09-06-2026	Jonas Degn Langdahl Grann	Completed	Added section 2.4 about choosing the right model

References

Reference	Title	Author

Table of contents

1 Introduction.....4

2 Setting Up AGENTS.md4

2.1 Best Practices for AGENTS.md.....4

2.2 Practical Example.....7

2.3 Bootstrapping with /init8

2.4 Choosing the right Model.....8

3 Creating and Using Skills – SKILL.md.....9

3.1 Skill Directory Structure9

3.2 Writing the SKILL.md File.....10

3.2.1 Best Practices for SKILL.md10

3.2.2 Practical Example.....11

3.3 How Skills Are Loaded11

3.4 Controlling Skill Access12

4 Optimizing prompts and context.....12

4.1 Prompt Engineering.....12

4.1.1 Best Practices13

4.1.2 Practical Example.....13

4.2 Context Engineering14

4.2.1 Best Practices14

4.2.2 Practical Example.....14

5 Development best practices15

5.1 Planning Approaches15

5.1.1 Skill-Based15

5.1.2 Spec-Driven16

5.1.3 Planning Pattern (Decomposition).....17

5.2 Execution Patterns18

5.2.1 Reflection Pattern (Self-Correction).....18

5.2.2 Tool-Use Pattern18

5.2.3 Multi-Agent Pattern.....19

5.3 Choosing the Right Approach.....19

5.4 Code Validation20

1 Introduction

Agentic AI represents a fundamental shift in how software is built. Where previous generations of AI tooling offered conversational assistance, answering questions, generating snippets, summarising documents, agentic AI systems operate autonomously: they reason about a goal, plan multi-step execution, invoke tools, observe results, and iterate until the task is complete. The developer's role shifts from writing every line of code to steering, reviewing, and validating the output of an AI agent that can read files, run commands, query databases, and edit code across an entire codebase.

This shift is happening now. Frontier models have crossed the threshold for sustained autonomous task execution, with context windows exceeding one million tokens, multi-hour task horizons, and standardised tool-integration protocols like MCP making agent-to-tool connectivity plug-and-play. Industry data points toward 20-40% operating cost reductions in AI-centric organisations, and analyst forecasts predict that 40% of enterprise applications will embed task-specific agents by end of 2026. For a delivery-focused company like Netcompany, this is not a future trend to monitor - it is a capability to operationalise immediately.

However, speed without control is a liability. Agents can produce plausible but subtly wrong code, hallucinate APIs, silently change behaviour, and crawl into parts of a codebase they should not touch. The faster AI accelerates development, the more discipline is required in review, governance, and guardrails.

2 Setting Up AGENTS.md

AGENTS.md is a project-level instruction file that shapes how every agent behaves in your repository. It defines conventions, constraints, and commands that the agent obeys throughout every session, it is placed in the root of the repository and ensures common guidelines for the project. It is committed to Git and shared across the team, so every developer's agent follows the same rules. Agentic AI tools read AGENTS.md at the start of every session and treat it as a persistent system prompt extension.

Instructions in AGENTS.md should be direct and imperative rather than descriptive. The file defines conventions, constraints, and commands that the agent obeys throughout every session.

2.1 Best Practices for AGENTS.md

DOs and DONTs

Introduce DOs and DONTs, but keep them small and relevant to the current directory scope.

```
### Do

- use MUI v3. make sure your code is v3 compatible
- use emotion css={{}} prop format
- use mobx for state management with useLocalStorage
- use design tokens from DynamicStyles.tsx for all styling. no hard coding
- default to small components. prefer focused modules over god components
- default to small files and diffs. avoid repo wide rewrites unless asked

### Don't

- do not hard code colors
```

- do not use divs if we have a component already
- do not add new heavy dependencies without approval

Why it helps: version specificity removes subtle bugs, state choice removes guessing, small components and diffs keep changes readable and reviewable.

Scoped Commands

Specify exactly the command formats to run builds for changed packages only. Fast feedback loops mean cheaper and more reliable agent operation.

```
### Commands

# Type check a single file by path
npm run tsc --noEmit path/to/file.tsx

# Format a single file by path
npm run prettier --write path/to/file.tsx

# Lint a single file by path
npm run eslint --fix path/to/file.tsx

# Unit tests
npm run vitest run path/to/file.test.tsx

# Full build when explicitly requested
yarn build:app
```

Note: Always lint, test, and typecheck updated files. Use project-wide build sparingly.

Safety and Permissions

Be explicit about what the agent can and cannot run without asking.

```
### Safety and permissions

Allowed without prompt:

- read files, list files
- tsc single file, prettier, eslint
- vitest single test

Ask first:

- package installs
- git push
```

- deleting files, chmod
- running full build or end to end suites

Project Structure Hints

Agents can search, but a few pointers save a lot of time from the agent having to re-explore your codebase with each new session.

- ```
Project structure
```
- see App.tsx for routes
  - see AppSideBar.tsx for the sidebar
  - components live in app/components
  - design tokens live in app/lib/theme/tokens.ts

### Use Concrete Examples

Examples beat abstractions. Point to real files that show your best patterns. Also call out legacy files to avoid.

- ```
### Good and bad examples
```
- avoid class-based components like Admin.tsx
 - prefer functional components with hooks like Projects.tsx
 - forms: copy app/components/DashForm.tsx
 - charts: copy app/components/Charts/Bar.tsx
 - data grids: copy app/components/Table.tsx
 - data layer: use app/api/client.ts for HTTP. do not fetch directly inside components

Why it helps: reduces drift from old code and increases fidelity (the agent mirrors your best examples in new code).

API Docs References

If you want a generated screen to work against real data on the first pass, show the agent where the docs and typed clients live.

- ```
API docs
```
- docs live in ./api/docs/\*.md
  - list projects - GET /api/projects using the typed client in app/api/client.ts
  - update project name - PATCH /api/projects/:id via client.projects.update

### Nesting AGENTS.md Files

Large repos benefit from hierarchical rules. Add an AGENTS.md in each critical subdirectory so guidance matches its exact stack and version. The agent reads the closest file to the work it is doing.

### PR Checklist

Be explicit about what "ready" means. This should be short and mechanical.

- ```
### PR checklist
```
- title: feat(scope): short description

- lint, type check, unit tests - all green before commit
- diff is small and focused. include a summary of what changed and why
- remove any excessive logs or comments before sending a PR

When Stuck, Plan First

Give the agent an escape hatch. If it is unsure, it should ask or propose a plan instead of guessing.

When stuck

- ask a clarifying question, propose a short plan, or open a draft PR with notes
- do not push large speculative changes without confirmation

Test First Mode (Optional)

On trickier tasks you may want the agent to create or update tests first, then code until green.

Test first mode

- when adding new features: write or update unit tests first, then code to green
- prefer component tests for UI state changes
- for regressions: add a failing test that reproduces the bug, then fix to green

2.2 Practical Example

The following AGENTS.md is adapted from the Temporal Java SDK and showcases the best practices described above.

Repository Layout

- temporal-sdk: core SDK implementation.
- temporal-testing: utilities to help write workflow and activity tests.
- temporal-test-server: in-memory Temporal server for fast tests.
- temporal-serviceclient: gRPC client for communicating with the service.
- temporal-kotlin: Kotlin DSL for the SDK.

General Guidance

- Avoid changing public API signatures. Anything under an internal directory is not part of the public API and may change freely.
- The SDK code is written for Java 8.

Building and Testing

1. Format the code before committing:


```
./gradlew --offline spotlessApply
```

2. Run the tests (prefer individual tests for speed):

```
./gradlew :temporal-sdk:test --offline --tests "io.temporal.workflow.*"
./gradlew :temporal-sdk:test --offline --tests "<package.ClassName>"
```

3. Build the project:

```
./gradlew clean build
```

Review Checklist

- ./gradlew spotlessCheck must pass.
- All tests from ./gradlew test must succeed.
- Add new tests for any new feature or bug fix.
- Update documentation for user-facing changes.

2.3 Bootstrapping with /init

Many agentic AI tools (including OpenCode) can generate an initial AGENTS.md for your project:

1. Open your project in your agentic AI tool.
2. Run the /init command (or equivalent).
3. The tool scans the project structure and creates an AGENTS.md at the project root.

IMPORTANT: Always review and refine the generated AGENTS.md before committing. Research has shown that auto-generated rule files perform worse than having no file at all if left unedited.

The generated file focuses on build, lint, and test commands, architecture and repo structure, project-specific conventions, and references to existing instruction sources.

2.4 Choosing the right Model

Feniks Build provides access to several models with different capabilities and cost profiles. Selecting the right model for the task at hand improves both the quality of results and the efficient use of your project's API budget.

Generally, match the model to the complexity of the task:

Task type	Recommended model type	Model example
Simple, well-defined tasks (e.g. renaming, formatting, small fixes)	Use a fast model optimized for coding for optimal speed and results.	Qwen 3.6-code (on-premise)
Moderate tasks (e.g. implementing a single feature following a plan, writing tests)	Use a fast model optimized for coding for optimal speed and results.	Qwen 3.6-code (on-premise)

Complex tasks, large skills, or multi-step agentic workflows (creating plans)	Usually, a large reasoning model is needed to plan more complex tasks. To avoid runaway costs use large reasoning models for planning only	Claude Sonnet 4.6 (Use for plan only)
Architectural reasoning, cross-cutting refactors, or planning of ambiguous problems	Usually, a large reasoning model is needed to plan more complex tasks. To avoid runaway costs use large reasoning models for planning only	Claude Sonnet 4.6 (Use for plan only)

Optimized models for coding, like Qwen 3.6 are hosted on Gefion or in Netcompany data centres, and are marked with “On Premise” in the Feniks Build documentation. For straightforward tasks these performs comparably with large reasoning models, making it the right default for most day-to-day work. Switch to a large reasoning model (like Claude Sotten) when the task requires deeper reasoning, when you are working with extensive skill files that demand more context comprehension, or when the cost of a wrong output is high. Refrain from using large reasoning models for the actual implementation, but use it for planning tasks only.

You can change the model at any time during a session — in the Desktop App via the model selector dropdown, or in the TUI via /models or F2. Model selection applies to the current session only and does not affect other open sessions.

Note: For guidance on which models are available and their associated costs, refer to the Feniks model catalogue or contact your project lead.

3 Creating and Using Skills – SKILL.md

Skills encode tested, project-specific workflows with defined steps, expected inputs, and validation logic. They turn the agent from a general-purpose assistant into a project-specific expert that follows your team’s proven procedures.

The project should maintain a set of skills that reflect their architecture and patterns. Each skill is a directory containing a SKILL.md file with YAML frontmatter and a markdown body with instructions.

3.1 Skill Directory Structure

Place skills in one of the following locations (listed in priority order):

Path	Scope
.opencode/skills/<name>/SKILL.md	Project (highest priority)
.claude/skills/<name>/SKILL.md	Project (Claude-compatible)
.agents/skills/<name>/SKILL.md	Project (agent-compatible)
~/.config/opencode/skills/<name>/SKILL.md	Global
~/.claude/skills/<name>/SKILL.md	Global (Claude-compatible)
~/.agents/skills/<name>/SKILL.md	Global (lowest priority)

For project-local paths, the agent walks upward from the current working directory to the git worktree root and loads any matching skills along the way. If duplicate skill names exist across locations, the project-local version takes precedence.

3.2 Writing the SKILL.md File

The SKILL.md frontmatter recognises these fields:

- **name (required): 1-64 characters, lowercase alphanumeric with single hyphens.**
- **description (required): minimum 20 characters, describes what the skill does and when to use it.**
- **license (optional): e.g. MIT, Proprietary.**
- **compatibility (optional): e.g. opencode.**
- **metadata (optional): string-to-string key-value pairs.**

3.2.1 Best Practices for SKILL.md

The following best practices are important information that all developers working with skills must read and understand.

Be concise. The context window is a shared resource. Your skill shares it with the system prompt, conversation history, other skills' metadata, and the user's request. Only add context the agent does not already have. Challenge each piece of information: does the agent really need this explanation?

Appropriate degrees of freedom. Match the level of specificity to the task's fragility and variability. High freedom (text-based instructions) for tasks where multiple approaches are valid. Medium freedom (pseudocode or scripts with parameters) when a preferred pattern exists. Low freedom (specific scripts, few parameters) for fragile operations like database migrations.

Write effective descriptions. The description field enables skill discovery. Always write in third person. Be specific and include key terms for both what the skill does and when to use it. The agent uses the description to choose the right skill from potentially hundreds of available skills.

Progressive disclosure patterns. Keep the SKILL.md body under 500 lines for optimal performance. Split content into separate reference files when approaching this limit. All reference files should link directly from SKILL.md (one level deep).

Use workflows for complex tasks. Break complex operations into clear, sequential steps. For particularly complex workflows, provide a checklist that the agent can copy into its response and track progress against.

Implement feedback loops. The pattern of "run validator, fix errors, repeat" dramatically improves output quality. Instruct the agent to validate its own output before declaring the task complete.

Start from real expertise. Effective skills are grounded in real project knowledge. Two practical approaches: (1) complete a real task in conversation with the agent, providing corrections along the way, then extract the reusable pattern; (2) feed internal documentation into the LLM and ask it to synthesise a skill.

Provide defaults, not menus. When multiple tools or approaches could work, pick a default and mention alternatives briefly.

Favour procedures over declarations. A skill should teach the agent how to approach a class of problems, not what to produce for a specific instance.

Plan-validate-execute for destructive operations. For batch or destructive operations, have the agent create an intermediate plan, validate it against a source of truth, and only then execute.

Bundle reusable scripts. If the agent independently reinvents the same logic each run, write a tested script once and bundle it in a scripts/ subdirectory within the skill folder.

3.2.2 Practical Example

The following example shows a skill for creating REST endpoints in a .NET project:

File: .opencode/skills/amplio-create-rest-endpoint/SKILL.md

```
---
name: amplo-create-rest-endpoint
description: "Create a new REST API endpoint in an Amplio .NET project using
  the CQS pattern, from domain entity to controller, with EF Core and OpenAPI
  configured."
license: Proprietary
compatibility: opencode
---

## Steps

1. Create the domain entity in the Domain layer.
2. Create the query/command and handler in the Application layer.
3. Add the DbSet to the DbContext.
4. Add the controller action in the appropriate Controller.
5. Return 404 NotFound if the entity does not exist.
6. Run dotnet build and dotnet test. Fix any errors before finishing.
7. Summarise the files created or modified.
```

3.3 How Skills Are Loaded

Skills are loaded on demand, not injected at session start. The agentic AI tool lists all available skills in the internal skill tool description. The agent sees each skill name and description, then decides whether to load the full content based on the current task.

You can invoke a skill explicitly by referencing it in your prompt, for example: "Use the amplo-create-rest-endpoint skill to create a Customer endpoint." Most tools also offer a /skills command (or equivalent) to browse available skills.

3.4 Controlling Skill Access

By default, all discovered skills are available to all agents. You can restrict access in your tool's configuration file (e.g. `opencode.json`):

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "skill": {
      "*": "allow",
      "experimental-*": "deny"
    }
  }
}
```

To restrict skills for specific built-in agents:

```
{
  "$schema": "https://opencode.ai/config.json",
  "agent": {
    "plan": {
      "tools": {
        "skill": false
      }
    }
  }
}
```

4 Optimizing prompts and context

4.1 Prompt Engineering

Prompt engineering is the practice of crafting inputs to an AI model to reliably produce accurate, useful, and consistent outputs. For agentic systems, where the model not only generates text but also plans, calls tools, and executes multi-step tasks, the quality of your prompts directly determines the reliability and cost-efficiency of the entire pipeline.

For deeper learning, refer to the Netcompany agentic AI e-learning modules and training materials.

4.1.1 Best Practices

Be explicit about the task and its boundaries. Agents interpret ambiguous instructions liberally. Always specify what the agent should do, what it must avoid, and what "done" looks like. Instead of "fix the bug", write: "Identify and fix the null-pointer exception in OrderService.java. Do not change the public API surface or modify existing tests."

Assign a role or persona in the system prompt. Framing the model as a domain expert activates relevant knowledge and raises output quality. Use a system prompt such as: "You are a senior Java developer working on a Spring Boot microservices project. You follow the conventions defined in AGENTS.md."

Provide concrete examples (few-shot prompting). Providing one or two examples of the desired input/output pair is one of the most reliable techniques for steering model behaviour. Point to real files in the codebase as "good examples" and explicitly name files or patterns to avoid.

Encourage step-by-step reasoning (chain-of-thought). For complex tasks, instruct the model to reason before acting: "First analyse the current implementation, then propose a plan, and only then make changes." This surfaces the model's assumptions early.

Specify the expected output format. Specify the required format of the output, especially when the result will be consumed by another system or agent. In multi-agent pipelines, unstructured prose from an intermediate agent will reliably degrade downstream reliability.

Build in a self-validation loop. Instruct the agent to validate its own output before declaring the task complete: "After making changes, run the linter and unit tests. Fix any new failures before finishing." This is one of the highest-leverage techniques available.

Avoid prompt injection. When agents read external data, that data may contain adversarial instructions. Never concatenate raw external input directly into the system prompt. Treat all externally retrieved content as data to be processed, not as instructions to be followed.

4.1.2 Practical Example

Weak prompt (vague, no constraints):

Add a GET endpoint for customers.

This will result in inconsistent output: the agent may choose any file, any pattern, any naming convention.

Strong prompt (explicit, bounded, with validation):

You are a senior .NET developer on the Amplio platform. Follow the CQS pattern and the conventions

in AGENTS.md. Do not modify any existing controller or test file.

Task: Add a GET /api/customers/{id} endpoint that retrieves a Customer by integer ID.

The Customer entity has: int Id, string Name, string Email, DateTime CreatedAt.

Steps:

1. Create the query and handler in the Application layer.
2. Add the controller action in CustomerController.cs.
3. Return 404 NotFound if the customer does not exist.

4. Run ``dotnet build`` and ``dotnet test`` and fix any errors before finishing.
5. Summarise the files created or modified.

This prompt produces a predictable, production-ready result because it establishes a role, references existing conventions, imposes clear constraints, decomposes the task into ordered steps, and closes the loop with a self-validation requirement.

4.2 Context Engineering

While prompt engineering focuses on how you phrase instructions, context engineering focuses on what surrounds those instructions. The context window is a finite and expensive resource: filling it with irrelevant files or stale history degrades reasoning quality and increases cost. Filling it with the right, highly relevant information is often the single most impactful improvement you can make to agent performance.

A useful mental model: think of the context window as a whiteboard in a meeting room. You want it to show exactly the information relevant to today's decision, not last month's diagrams and not the entire project archive.

For deeper learning, refer to the Netcompany agentic AI e-learning modules and training materials.

4.2.1 Best Practices

Clear your session for unrelated tasks. Start fresh when beginning a new task. Most agentic AI tools provide a `/new` command (or equivalent) to reset your session.

Use `/compact` to summarise history. As an agentic workflow progresses, raw conversation history grows rapidly. Most tools offer a `/compact` command (or equivalent) to replace the full history with a compact summary.

Inject fragments, not entire files. Passing a 2,000-line source file when the agent only needs one class wastes tokens. Use `@` to reference specific files. Every token in the context window should be earning its place.

Leverage subagents for exploratory work. When the agent needs to research or explore the codebase, let it spawn a subagent. The subagent's context is isolated, so the exploration does not accumulate in the main session.

Treat external data as data, not as instructions. Content retrieved from files, databases, or web pages can contain adversarial text (prompt injection). Maintain a clear separation between the instruction layer and the data layer.

4.2.2 Practical Example

Naive approach (over-filled context):

System prompt: 5,000 tokens (role + all global AGENTS.md rules)

Retrieved context: 450,000 tokens (entire src/ directory injected)

History: 20,000 tokens (full raw conversation)

Task prompt: 1,000 tokens

Total: 476,000 tokens | Result: slow, expensive, unfocused

Engineered approach (focused context):

System prompt: 2,000 tokens (role + submodule AGENTS.md only)

Retrieved context: 18,000 tokens (changed files + dependencies + tests + PR description)

History: 1,500 tokens (summarised: 3 prior review comments)

Task prompt: 500 tokens

Total: 22,000 tokens | Result: fast, cheap, precise

The engineered approach is 95% smaller, faster, cheaper, and produces more specific, actionable output.

5 Development best practices

The first, and maybe most important principle to adhere to when using Agentic AI in [PROJECT] is to understand that you are still accountable for all the code produced. You must be able to understand and validate the correctness. All our quality principles still count: The author is responsible for writing the code, the reviewer must review and understand the code, and we are accountable for the final result deployed to production.

This section describes the recommended methodology for completing tasks with an agentic AI tool. Whether you are implementing a feature, fixing a bug, or refactoring code, the overall pattern is the same: start by understanding the problem in Plan mode, choose an approach that matches the task's complexity, and then execute with the appropriate level of agent autonomy.

Every task begins with discovery. Before writing a single line of code, switch to the Plan agent and prompt it to read and analyse the relevant parts of the codebase. Planning mode prevents accidental changes while you and the agent build a shared understanding of the problem.

5.1 Planning Approaches

There are three primary ways to frame a task for the agent. The right choice depends on whether you have a reusable workflow, a clear specification, or a complex goal that needs decomposition.

5.1.1 Skill-Based

Use this approach when a reusable skill exists for the type of work you need done.

When to use it:

- A SKILL.md already exists for the task.
- The task follows a repeatable pattern that benefits from consistency across the team.
- You want the agent to follow a specific sequence of steps rather than deciding its own approach.

How to apply it:

4. Identify the skill that matches your task.
5. Write your prompt around the skill, providing the inputs the skill expects. Reference the skill explicitly by name.
6. Let the agent load the skill and follow its workflow. Review the output against the skill's validation steps.

Example prompt:

Use the `amplio-create-rest-endpoint` skill to create a new endpoint.

Entity: Invoice

```
Properties: int Id, string InvoiceNumber, decimal Amount, DateTime IssuedAt
```

```
ID type: int
```

```
Feature verb: Get
```

```
Base namespace: Netcompany.Amplio
```

```
DbContext namespace: Netcompany.Amplio.Infrastructure
```

Tip: If you find yourself repeatedly correcting the agent on the same type of task, that is a signal to extract the corrections into a new skill.

5.1.2 Spec-Driven

Use this approach when you have a clear set of requirements and acceptance criteria but no existing skill covers it.

When to use it:

- You have well-defined requirements from a user story, ticket, or technical design.
- The task is not a repeatable pattern worth encoding as a skill.
- You want the agent to make implementation decisions within clear boundaries.

How to apply it:

7. Define the requirements as precisely as possible.
8. State the acceptance criteria explicitly.
9. Specify what the agent must not change.
10. Include a validation step.

Example prompt:

You are a senior .NET developer on the Amplio platform.

Follow the CQS pattern and the conventions in AGENTS.md.

Requirements

Add a soft-delete feature to the Invoice entity.

- Add a nullable DateTime property DeletedAt to the Invoice entity.
- When an invoice is deleted via DELETE /api/invoices/{id}, set DeletedAt to UTC now instead of removing the row.
- The existing GET /api/invoices endpoint must exclude soft-deleted invoices by default.
- Add a query parameter ?includeDeleted=true to override this filter.

Acceptance criteria

- DELETE /api/invoices/{id} returns 204 and sets DeletedAt.


```

- GET /api/invoices returns only non-deleted invoices.
- GET /api/invoices?includeDeleted=true returns all invoices.
- Calling DELETE on an already-deleted invoice returns 404.
- All existing tests still pass.
- New unit tests cover the soft-delete behaviour.

## Constraints
- Do not modify any existing controller tests.
- Do not change the public API signature of existing endpoints.
- Run dotnet build and dotnet test and fix any errors before finishing.

```

5.1.3 Planning Pattern (Decomposition)

Use this approach when the task is too complex to specify upfront, or when you are not sure what the right implementation approach is.

When to use it:

- The task is large or ambiguous and you need to explore before committing.
- You want the agent to read the codebase and propose a plan based on what it finds.
- The task spans multiple files, layers, or concerns.
- You want to catch architectural mistakes before any code is written.

How to apply it:

11. Switch to the Plan agent.
12. Describe the goal at a high level.
13. Ask the agent to produce a numbered plan. Review, give feedback, iterate.
14. Switch back to Build and instruct it to execute the approved plan.

Example prompt (in Plan mode):

```

We need to add multi-tenancy to the Invoice module.

Each tenant has a TenantId (Guid) and invoices must be scoped
to the tenant of the authenticated user.

Analyse the current Invoice implementation and propose a
step-by-step plan for adding tenant isolation. Consider:
- Where to add TenantId (entity, DB, queries, controllers)
- How to extract TenantId from the auth context
- How to ensure no cross-tenant data leakage

```

- Which existing tests need to be updated
- What new tests are needed

Do not make any changes. Only produce the plan.

The decomposition pattern is especially valuable for tasks where a wrong architectural choice early on would be expensive to reverse.

5.2 Execution Patterns

The planning approaches above describe how you frame a task. The patterns below describe how the agent carries out the work. These patterns complement any planning approach and can be combined.

5.2.1 Reflection Pattern (Self-Correction)

The reflection pattern forces a self-review loop. Instead of accepting the agent's first output, you instruct it to generate a draft, critique its own work, and fix its own errors before returning a final result.

This is one of the highest-leverage patterns available. Agents with a validation loop produce significantly better output than agents that generate once and stop.

How to apply it:

After making changes:

1. Run dotnet build and fix any compilation errors.
2. Run dotnet test and fix any failing tests.
3. Run the linter (dotnet format --verify-no-changes) and fix violations.
4. Review your own changes against the conventions in AGENTS.md.
5. Only report completion when all checks pass.

The reflection pattern can also be embedded in skills or in AGENTS.md as a global completion checklist.

5.2.2 Tool-Use Pattern

Instead of providing all context upfront in your prompt, give the agent a sandbox of tools and let it decide how to gather information and execute the task dynamically.

Example prompt:

Find all usages of the deprecated PaymentGateway class across the codebase.

For each usage, determine whether it can be replaced with the new PaymentService class. Propose a migration plan, then execute it.

Run the tests after each file change.

The tool-use pattern becomes more powerful as you give the agent better tools (MCP servers, skills). Boundaries defined in AGENTS.md are essential to prevent the agent from exploring too broadly.

5.2.3 Multi-Agent Pattern

The multi-agent pattern divides a large task by delegating specialised subtasks to separate agents, each with its own context window, tools, and system prompt.

Automatic delegation:

```
Investigate how error handling is implemented across the codebase,
then use that information to add consistent error handling to the
OrderController. Follow the patterns you find in the best-implemented
controllers.
```

Explicit delegation:

```
@explore find all API controllers and describe the error handling
pattern used in each. Note any inconsistencies.
```

Custom subagents:

For recurring multi-agent workflows, define custom subagents with specialised system prompts:

```
---
description: "Reviews code changes for correctness and convention adherence."
mode: subagent
model: anthropic/claude-sonnet-4-20250514
temperature: 0
tools:
  write: false
  edit: false
---

You are a code reviewer. You cannot modify files.

Review the changes made in this session against AGENTS.md conventions.

Report issues as a numbered list with file, line, and description.
```

5.3 Choosing the Right Approach

The approaches and patterns described above are not mutually exclusive. Most real tasks combine a planning approach with one or more execution patterns:

- Small, repeatable tasks: Use the skill-based approach. If no skill exists yet and you find yourself doing it more than twice, create one.
- Well-defined features with clear requirements: Use the spec-driven approach with the reflection pattern.
- Large or ambiguous tasks: Start with the planning pattern in Plan mode. Once you have a plan, switch to Build and execute with the reflection pattern.

- Tasks spanning many files or modules: Use the tool-use pattern combined with the multi-agent pattern.
- Critical or high-risk changes: Combine decomposition with reflection and multi-agent (have a review subagent check the work).

The common thread is that better preparation produces better results. Time spent in Plan mode, writing clear specs, or choosing the right skill is always repaid in fewer review cycles and less rework.

5.4 Code Validation

Code generated by AI must be validated by the developer who generates it. The developer still owns every single line of code written and has the responsibility to validate its correctness.

IMPORTANT: AI is a powerful tool, but it is not meant to replace your skills and expertise. Use it to supplement your abilities. Be careful with becoming too dependent on its suggestions. Remember, the human touch is still irreplaceable, and true mastery lies in the harmonious blend of human ingenuity and AI's computational power.